

ULTRON — Full Repository Analysis & Development Handoff

You are now working on my personal AI project called **ULTRON**.

I am providing you with: 1. The **current ULTRON repository/codebase** 2. A text document containing the current ULTRON architecture, design decisions, intended behavior, and project status.

IMPORTANT — DO NOT MODIFY CODE YET

Your first job is **analysis, not implementation**.

Do NOT:

- rewrite the architecture
- delete files
- rename files
- replace working components
- add unnecessary frameworks
- create duplicate systems
- make large changes based only on assumptions

First understand what already exists.

1. Understand the project

Read the supplied ULTRON architecture document completely.

Then inspect the **entire repository**, including:

- Python files
- packages
- configuration
- tests
- data files
- documentation
- imports
- entry points
- dependencies

Do not assume that the documentation is 100% accurate.

The **actual code is the source of truth for what is currently implemented.**

The architecture document is the source of truth for the intended design.

Your job is to compare the two.

2. Create a complete file-by-file map

For EVERY important file, determine:

- File path
- Main purpose
- Classes
- Functions
- Inputs
- Outputs
- Dependencies
- Files that call it
- Files it calls
- What type of user request it handles
- What it should NOT handle
- Current implementation status
- Problems/bugs
- Whether it matches the intended architecture

Use a structure like:

FILE: PURPOSE: MAIN CLASSES/FUNCTIONS: INPUT: OUTPUT: HANDLES: DOES NOT HANDLE: CALLED BY:
CALLS: DEPENDENCIES: CURRENT STATUS: PROBLEMS: ARCHITECTURE MATCH: RECOMMENDED ACTION:

3. Map the complete ULTRON flow

Determine the ACTUAL execution flow from user input to final response.

For example, investigate whether the real system currently behaves like:

USER ↓ INPUT PROCESSING ↓ CORE 1 ↓ ROUTER |—— COMMAND |—— PIPELINE 1 |—— PIPELINE 1.5
|—— PIPELINE 2 ↓ CORE 2 / VERIFICATION ↓ ULTRON TOUCH ↓ FINAL RESPONSE

Do NOT assume this flow is correct.

Trace the actual imports, function calls, and execution paths and report the real flow.

4. Analyze Core 1

Determine exactly what Core 1 currently does.

Test/analyze its handling of different request types, including examples such as:

- "Hello"
- "What is gravity?"
- "Explain gravity"
- "Explain gravity briefly"
- "Explain gravity in detail"
- "Who is Albert Einstein?"
- "What is Python?"
- "How does a CPU work?"
- "Calculate 25×37 "
- "Search for the latest news"
- "Open Notepad"
- "Write a Python program"
- complex multi-step requests

Determine:

- intent classification
- complexity classification
- route selection
- response mode
- confidence
- edge cases
- incorrect classifications
- fallback behavior

Do not change Core 1 yet.

5. Analyze the Router

Determine exactly how the Router works.

Document:

- Every possible route
- Conditions for each route
- Default/fallback route
- Invalid route behavior

- Error handling
- Escalation behavior
- Whether routing actually matches Core 1's output
- Whether routing matches the architecture document

Pay special attention to contradictions involving:

- Pipeline 1
- Pipeline 1.5
- Pipeline 2
- Command execution
- Complexity levels

6. Analyze Pipeline 1, Pipeline 1.5 and Pipeline 2

For each pipeline determine:

- Its actual purpose
- What requests it receives
- What it does
- What model/tools it uses
- What it returns
- Whether it verifies its result
- What happens if it fails
- Whether it can escalate to another pipeline
- Whether its behavior matches the architecture document

Create a comparison table:

PIPELINE | INTENDED PURPOSE | ACTUAL PURPOSE | INPUT | OUTPUT | STATUS | PROBLEMS

7. Analyze Core 2 / verification

Determine whether Core 2 is actually implemented and how it works.

Check:

- verification
- confidence
- hallucination detection
- correction/improvement
- approval/rejection
- retry/escalation

- interaction with pipelines

Determine whether Core 2 is a genuine verification layer or only partially implemented.

8. Analyze Research

Inspect the research system.

Determine:

- search providers
- retrieval
- filtering
- relevance checking
- source handling
- research quality
- failure handling
- how research results reach the final answer

Check whether Pipeline 1.5 actually performs the live-research behavior described in the document.

9. Analyze Memory

Inspect all memory-related files.

Determine:

- short-term memory
 - persistent memory
 - conversation context
 - storage format
 - read/write behavior
 - when memory is accessed
 - whether memory is actually connected to ULTRON
 - whether persistent memory is implemented or only planned
-

10. Analyze Tools and Commands

Inspect the command/tool system.

Determine:

- available commands
- tool execution
- safety checks
- OS interaction
- error handling
- command routing
- whether normal questions can accidentally reach command execution

Do not add dangerous or destructive capabilities.

11. Analyze Ultron Touch / final response processing

Determine exactly what happens between the raw pipeline result and the final user response.

Check:

- formatting
 - cleanup
 - personality
 - unnecessary disclaimers
 - response modes
 - whether information is accidentally modified
 - whether important content can be lost
-

12. Find contradictions and broken architecture

Create a section called:

ARCHITECTURE CONTRADICTIONS

Find every case where:

- documentation says one thing but code does another
- two files have overlapping responsibilities
- Core 1 produces a value the Router doesn't understand
- Router sends requests somewhere unexpected
- pipelines return incompatible formats
- Core 2 expects a different format
- final response processing expects something different

- imports/dependencies are inconsistent

Do not silently fix these.

Report them first.

13. Find technical problems

Look for:

- broken imports
- circular imports
- unreachable code
- unused files
- duplicate logic
- inconsistent naming
- hard-coded values
- incorrect defaults
- exception handling problems
- API/model configuration problems
- malformed return structures
- missing tests
- dead code
- inconsistent interfaces

Separate REAL problems from merely stylistic suggestions.

14. Test the existing system

If the environment allows it, run the existing tests.

Also run safe basic tests of the main ULTRON flow.

Do NOT perform destructive OS actions.

Record:

TEST: EXPECTED: ACTUAL: PASS/FAIL: ERROR:

Do not modify code just to make tests pass during this analysis phase.

15. Produce a final audit report

At the end, give me:

A. PROJECT OVERVIEW

Explain the current ULTRON architecture in simple terms.

B. FILE MAP

Complete important-file responsibility map.

C. ACTUAL EXECUTION FLOW

Show how a request really travels through the code.

D. QUESTION → ROUTE MAP

Create examples such as:

USER REQUEST → CORE 1 DECISION → ROUTER → PIPELINE/COMMAND → VERIFICATION → FINAL RESPONSE

Use real behavior from the code.

E. IMPLEMENTED FEATURES

List what genuinely works.

F. PARTIALLY IMPLEMENTED FEATURES

List what exists but is incomplete.

G. PLANNED BUT NOT IMPLEMENTED

List features described in the document but missing from the code.

H. BUGS

List actual bugs separately from improvements.

I. ARCHITECTURE CONTRADICTIONS

List every contradiction between documentation and implementation.

J. PRIORITY PLAN

Give me a development plan:

P0 — Critical P1 — Important P2 — Improvements P3 — Future features

K. RECOMMENDED NEXT STEP

Tell me exactly what should be fixed or implemented FIRST and WHY.

16. VERY IMPORTANT DEVELOPMENT RULE

ULTRON is a long-term project.

Do not turn it into a generic chatbot.

Preserve the modular architecture and separation of responsibilities.

Prefer:

Core 1 → Router → Specialized processing → Verification → Final response

rather than putting everything into one giant AI call.

Use deterministic/local logic where appropriate and AI/LLM reasoning where appropriate.

Do not introduce complexity unless it provides a real benefit.

17. After the audit

STOP.

Do not modify the repository yet.

Wait for my approval.

After I review your audit, we will work on ULTRON **one subsystem at a time**, with tests after each significant change.

The first objective is not to make ULTRON bigger.

The first objective is to make the **existing architecture correct, connected, testable, and reliable**.

Remember:

DOCUMENTATION = intended architecture CODE = actual implementation YOUR JOB = accurately compare the two before changing anything.